

Lec 25 Practical Exercise - 1

Lec 25 Practical Exercise - 1

Lecture	25 / 63 + intro
Duration	33:25
Captions	English (auto-generated)
Words	4,923
Video	5Iiqeoxw_Sk
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all. Welcome to the hands-on series on fundamentals of generative AI and large language models theory and practice. In today's session, we'll be learning the implementation of variational autoencoder, which is slightly different from the conventional autoencoders. As the name suggest, autoencoders will comprise of two parts, the encoding and the decoding part. So, what happens in the basic autoencoder is the encoder will act only on the input or the input images in our case and try to get a latent vector, which is a fixed vector, which cannot be changed. And when this vector is given to the decoder, the decoder gets back the reconstructed input or gets the reconstructed image.

[00:47] But in case of variational autoencoder, this encoder acts on the input image and also it makes use of the probability distribution of the input image. That is, input image will be having a set of pixel values. So, from those pixel values, we try to get the mean as well as the variance value and then find a latent vector that is Z , which is a dependent on both the mean, the variance and a small noise, which is called as epsilon. So, when this is given to the decoder, the decoder gets back the reconstructed image. So, let us see the implementation of such a variational autoencoder.

[01:28] So, in order to have this a variational autoencoder, we initially import the necessary libraries. So, the libraries required are a TensorFlow, Keras and from Keras we need layers, models and we need the package that is Matplotlib. So, importing TensorFlow as TF. TensorFlow is a deep learning framework, which is required to build the deep learning models. And from Keras we import layers as well as well as models. So, in order to develop a neural network, we need the input layers, we need a set of hidden layers, as well as output layers. That's why we need this layers.

[02:03] And to create or build or construct a model, we need this model function. And for data visualization, we need this matplotlib. From matplotlib.pyplot, we get this PLT. We call this as PLT. Now coming to the data set, we have worked on the MNIST data set, which is a handwritten digit data set comprising of 10 different classes, varying from digits 0 to 9. So, we have seen the types of autoencoders, as well as regularization techniques on those autoencoders on the same data set. So, in continuation of those autoencoders, we have this variational autoencoders.

[02:45] So, we are making use of the same MNIST data set comprising of 10 different classes. So, now we load this MNIST data set from Keras, and then store it into variables X_train and X_test. So, as we all know, we need a set of sample images for training, as well as test purpose. But in our case, we have only this X_train coming under training set, and we have this X_test coming under the test set. We don't require X We don't require Y_train or Y_test because it does not require the variational autoencoder does not require the output labels. So, we ignore that.

[03:27] We ignore this using the underscore. That's why we are using this underscore here. So, now coming to X_train, what comes under X_train and X_test? So, the new X_train and X_test will be the original training set and the test set, which is converted into floating data type, and then normalized in the range 0 to 1. So, what kind of normalization we follow is since this MNIST data set is a grayscale data set, the values vary from the range 0 to 255.

[03:59] So the minimum value what we have is 0 and the maximum value what we have is 255. That's why when we take an input image in our case, it is a grayscale image. So let me take an example of a 3 cross 3 image here. The values lie in the range 0 to 255. 16 2 So I'm randomly taking these values. So what happens is every value will be normalized in such a way that it is $10 / 255$, $0 / 255$, $1 / 255$. Likewise, all the values are divided by 255. So the minimum value is 0 that we obtain. And in case we have one of these values, say for example 255 here, this will be $255 / 255$. And the maximum value happens to be 1. So normalization is done in order to get the data in a proper range. In our case, it is 0 to 1.

[05:00] Now coming to X_train and X_test once again, we reshape this training set and test set so that we have only the input features. So the given MNIST data set, whatever data set we have taken, it is of the shape 28 cross 28 pixels. So we have

28 pixels as width, 28 pixels as height. And since it is a grayscale image, it is always the channel which is 1. So this 28 cross 28 will be now converted into 784 different input features. So So that's why we have this reshaping -1,784.

[05:41] So -1 is required in order to ignore the output label. So once this is done, let us print the shape of training data set as well as the test data set. So, when we print this shape, we get 60,000 samples in the training set having the input features as 784 and 10,000 samples in the test set having the input features as 784. So, this is all about loading the data set as well as the initial pre-processing on the data set.

[06:13] Now, coming to defining the sampling layer. So, as mentioned earlier, our variational autoencoder is different from the normal autoencoder because it has the encoding part or the encoder, which acts on the input X. Soon after which, we have the sampling layer which has to be defined. After the sampling layer, we have the decoding part, which is done by the decoder. So, in the sampling layer, what happens is we have to get a vector Z, which is dependent on the probability distribution. And this Z is dependent on the mean of the variables that we take plus the standard deviation into the epsilon value. So, the standard deviation is based on the variance of the data and epsilon is some random noise. We can

[07:09] take it as a Gaussian noise or a normally distributed noise, which has the mean zero and standard deviation as one. So, this epsilon is a a Gaussian noise, which has to be considered. So, in order to have the sampling layer, we have to define this vector Z, which is dependent on the mu value, mean, epsilon, as well as the standard deviation, which is obtained from the variance of the data. So, let me initially define the parameter that is latent dimension, let it be 16 in number. So, that we have the seven 784 input features. We have the 78 84 input features, which have to be converted into 16 latent variables.

[08:01] Now, from this 16 different latent variables, say I call it as X1, maybe X2, X3, so on up to X16. Under each of these variables, we will have values for every image or every image samples. Now, I need to take mu of the sample called as mu1, mu2, mu3, similarly, mu16. Similarly, I need to consider variance one, variance two, variance three, so on, variance 16. From this, I'll be getting the latent space that is Z. So, defining the parameter latent dimension is equal to 16. We are free to choose any dimension here. For simplicity, we have chosen 16.

[08:51] So, now creating the sampling layer from the set of inputs. The set of inputs are now divided into two values, that is a mean value as well as the variance value. So, uh and then this epsilon value, which is a Gaussian noise or a normally distributed noise, is got by random distribution or random sampling of the normally distributed data. And the shape of this epsilon is should be same as the shape of the mean that we are taking. So, mean value, suppose I consider uh the mean value of 16 latent variable, then I'll be having 16 different values. The same way, epsilon should be 16

in values.

[09:29] So, at the end of this, we return the mean value as well as the standard deviation, which is to be multiplied by epsilon. So, now what is this TensorFlow? From this TensorFlow, we take exponent of 0.5 into Z log of variance. So, this is nothing but the relation between standard deviation and the variance. So, we know that the standard deviation, which is represented as sigma, is square root of variance. So, variance is nothing but sigma squared.

[10:03] So, we have on the left-hand side, we have the sigma and then sigma squared, which can be also written as When we take log on both the sides, it is log, maybe log to the base e. So, log to the base e sigma is equal to half log to the base e sigma squared. And then we we need to derive the standard deviation. The standard deviation, which is required to be substituted in the Z equation. So, what will be the sigma? Sigma will be e power 1 by 2 log of sigma squared to the base e, which can be also written as e power 0.5 log of sigma squared to the base e. So, on the left-hand side, we have the standard deviation. So, this expression is been written in the form of code here.

[11:00] Exponential 0.5 into Z log variance into the epsilon plus the mean will give us the Z, that is the latent vector. So, this is all we define in the sampling layer. Now, we have the encoder followed by the sampling layer and then the decoder. So, decoder will give us the reconstruction of the image. So, what are the components coming under this encoder? So, to to the encoder, we give the input image of the shape 784 because we have 784 different pixel values, which is obtained by multiplying 28 pixels in width and 28 pixels in height. 28 into 28 will give us 784.

[11:42] Uh soon after this, we create a fully connected layer or a dense layer comprising 256 different neurons, and the activation function is ReLU activation function, which is acting on the encoder inputs. So, ReLU will try to give the maximum value. Uh so, what happens is if we have a negative value, it will give us zero. If we have a positive value, it will try to retain the same value. That is So, the functioning of ReLU is maximum of zero {comma} X. In our case, it is Z.

[12:26] of Z. So, when if we have a positive value, for example, if the value is 5, maximum of zero {comma} 5 will be equal to 5. Suppose we have some negative value, say minus 2, then maximum of zero and minus 2 is zero. So, this is how non-linearity will be added to the network. Yeah. Now, coming to finding the mean value and the variance value. So, how do we get it as it is based on the latent dimension, which is 16 variables, and the name given to this uh variable is a Z mean, which is got by taking the input image, that is X. Same way, the variance of the data or from which we want the standard deviation is got by uh taking the dense layer, which is the fully connected layer, which is acting on 16 different input variables,

[13:16] and let me give the name for this as Z log variance, which is acting on once again the input image. So, we are doing the random sampling of this mean as well

as the variance value. So, the encoder requires the encoded inputs, which is earlier defined here. Also, not this is nothing but X , from which we need the mean value, we need the standard deviation, also the Z value, which is dependent on the mean as well as standard deviation and the epsilon value. So, finally this architecture is called as the encoder architecture.

[13:54] So, we have constructed the encoding part here. Now coming to the decoder. So, at the decoding part, we have this 256 neurons because we have uh created 256 neurons in the encoding part. So, the same thing to be represented here. So, 256 neurons, activation function is once again ReLU to add non-linearity to the data. And what is it acting upon is the latent inputs. Now, what are these latent inputs? The latent inputs is nothing but the vector Z , which is dependent on the probability distribution of μ and the variance. So, that is completely dependent or the shape of that data is completely dependent on the latent dimension, which is 16 in number.

[14:41] So, now the decoder outputs are got by considering the fully connected layer by once again taking the same number of neurons as that of the input, that is 784. Let the activation function be sigmoid. So, what is the role of the sigmoid is it will try to give the values in between 0 to 1. Why we need to consider the sigmoid activation function is initially we have normalized the value. We have normalized the inputs in the range 0 to 1.

[15:14] Now, what is the sigmoid function doing? Always, it will try to give the value, say for example sigma of X will be equal to $1 / 1 + 1 / 1 + e^{\text{power } X}$. Whatever may be the value of X , maybe it can vary from minus infinity to to infinity, always the value range between 0 and 1. Suppose we substitute minus infinity, then we get the value as one. Suppose we have the value as infinity, we get the value as zero. So, always the values lies lie between 0 to 1, which is same as our normalized input. So, therefore, at the end we put this sigmoid activation function.

[16:00] So, now what is our decoder? It is nothing but a model which is acting on the latent inputs as well as the decoder outputs. And the name of this portion of the architecture is called as the decoder architecture. So, we have defined the encoder. In between encoder and decoder comes the sampling layer, which is already defined earlier. And then also we have defined the decoder. Now, coming to the construction or creation of this variational autoencoder model. So, we create this variational autoencoder model from the TensorFlow or the Keras model class.

[16:38] And then we define we do the initial definition of the encoder, the decoder, the inputs that have to be defined, or the sampling layer which has to be defined. So, throughout this code cell, it is all about creation of the variational autoencoder, and how is it different from the normal autoencoder. What are the loss functions, or what are the losses that it need to be considered, which makes it different from the normal autoencoder.

[17:07] So, we define the encoder, we define the decoder. So, encoder acts on the inputs in order to get the mean and the variance. And the decoder when acts upon this latent space or the latent vector Z gives us the reconstructed image. So, once this encoder part for is followed by the decoder part, we get the reconstructed image. So, what happens during the training here? As mentioned earlier, we need only the input data. Our variational autoencoder does not depend on the output label. So, uh we can consider this input as a tuple.

[17:50] So, tuple is something which consists of which can consist of multiple elements. So, in our case, suppose you we consider X_{train} and Y_{train} , which is the general way of taking the data. So, a tuple may consist of multiple elements. So, in this case, X_{train} as well as Y_{train} . But, we are interested only in considering the X_{train} value. That is why we consider data of zero. Our data can consist of both the input as well as output, but we need only the zeroth index. That is the first value what is present in this tuple.

[18:27] Similarly, if we represent the test data as X_{test} and Y_{test} , when we write data of zero, this this is our entire data, which is represented in the form of a tuple. So, when we write data of zero, it is only this X_{test} which is being considered. So, if is the instance data, if the data is a tuple, which is comprising of multiple elements, if the data is a tuple, then from the data we take only the zeroth index data, which is the training set or the input training set or the test training set.

[19:10] If if is instance data is a tuple, then from the data we have to take only the zeroth index. So, zeroth index may consist of only the input features. Maybe it is a training set or a test set, it can consist only of the input features. So, making use of those input features, what we do next is calculate the gradient. So, how do we calculate the gradient or why is this gradient important? Gradient is important in order to understand how much weight has to be modified in order for the loss to be very minimal. So, in case of autoencoders, we have seen a reconstruction loss, which is basic loss, which has to be very minimal. But, in case of this variational autoencoder, there are two different losses which

[20:02] come into picture. One is the reconstruction loss, other one is the KL divergence loss. So, this reconstruction loss can be sometimes the binary cross entropy loss or cross entropy loss or sometimes even the sparse cross entropy loss. But, KL divergence loss is the loss which is completely dependent on the variance as well as the mean of the probability distribution of the input variables. So, let us see how these two loss values has to be considered.

[20:32] So, let us see this reconstruction loss. So, the reconstruction loss will be equal to the binary cross entropy loss. So, what is this binary cross entropy loss? So, the total loss in a variational autoencoder comprises of the reconstruction loss plus the KL divergence loss. So, this reconstruction loss can be a binary cross entropy

loss. So, this BCE will be equal to minus $\sum Y \log(Y) + (1 - Y) \log(1 - Y)$. Y refers to the actual output.

[21:39] And $\hat{Y}$ refers to the predicted output. So, this is the binary cross entropy loss. Now, coming to the KL loss This will be $-\frac{1}{2} \sum (\log(\sigma^2) + \frac{(\mu - \hat{\mu})^2}{\sigma^2})$. So, this KL divergence loss completely depends on the variance that we have considered as well as the μ value, that is the mean of the probability distribution. So, when these two are put together, this is the total loss that is incurring in the variational autoencoder.

[22:21] So, this mathematical representation is written in the form of a code here. So, reconstruction loss will be uh the binary cross entropy loss, which is dependent on whatever data we have considered and whatever reconstructed data we have. So, data is the original input image, reconstruction is the data or the output image that is obtained after the decoding part. So, what we can do is binary cross entropy, we can take the mean of it.

[22:50] That's why uh we have this reduced value or we have the reduced loss value. So, what we try to do is we will have binary cross entropy loss one Similarly, binary cross entropy loss two for 16 different variables. For 16 different variables. So, what we do is each of these loss, say for example, if the loss one is 0.02, loss two is 0.04, say this is 0.16. All these loss will be added. They will be added and then divided by 16 different values.

[23:29] So, this gives us the mean reconstruction loss. So, now coming to the KL divergence loss, it is $-\frac{1}{2} \sum (\log(\sigma^2) + \frac{(\mu - \hat{\mu})^2}{\sigma^2})$. So, this is the KL divergence loss which is incurring in the variational autoencoder. So, total loss will be equal to reconstruction loss plus the KL loss. Now coming to calculation of gradients. How exactly the gradient can be calculated? So, what we have done is later part we we can see that we have used the optimizer Adam. So, the Adam optimizer makes use of the mean estimate as well as the variance estimate in order to calculate the gradients.

[24:19] Otherwise, the gradient the generalized form of this gradient can be written as new weight will be equal to the old weight minus $\eta \frac{\partial L}{\partial W}$. So, then updation of the weights or the new weights will be equal to the old weights minus η . η refers to the learning rate of the model. And $\frac{\partial L}{\partial W}$, that is uh variation of loss with respect to variation of weights will be equal to our gradient. So, $\frac{\partial L}{\partial W}$. So, this gradient is completely dependent on how much you we need to adjust the weight based on whatever loss that is happening.

[25:07] So, this kind of training we need to give. So, in the forward forward pass, we need to calculate the weights, bias, and then the Z vector, but in the backward propagation, we need to calculate this gradient. So, that is done in this part of the code cell. At at the end, we have to return the total loss, which is sum of reconstruction loss as well as the KL loss. So, this is all about how we create a variational auto

encoder.

[25:41] So, variational auto encoder comprising of two parts, the encoder and the decoder. When we configure, we make use of some optimizer. In our case, it is Adam. So, we are using Adam here because it is dependent on the mean as well as the variance estimate. Otherwise, we are free to use any optimizers, maybe stochastic gradient descent or gradient descent with momentum or Adagrad, RMSprop, and so on. But, in our case, since we make use of mean as well as variance, we are sticking onto this Adam optimizer.

[26:14] So, now coming to training the variational auto encoder. So, we have to fit this model onto the training data, which is X_{train} , and let the number of epochs be equal to 10. Let the batch size be equal to 128. So, why we need to convert this entire training set into batches is for the updation of the weights. Let the updation of the weights happen at the end of every 128 different samples. Let us not wait until the end of 50,000 samples and then upgrade the weights. So, in that case, the loss will be very much maximum. So, in order to have a control over the loss value, we are uh the input data or the training data into several batch size.

[26:55] So, uh we have this variational autoencoder training on the X_{train} data. So, let us see what is happening here. At epoch number one, we are calculating the KL divergence loss, the total loss, as well as the reconstruction loss. Similarly, epoch two, uh we have the KL loss, the total loss, as well as the reconstruction loss. So, at the end of 10th epoch, we can see that the KL loss is 3.4×10^{-5} . Uh the loss, the total loss is 0.2644, and the reconstruction loss is 0.2643.

[27:35] So, summing of this reconstruction loss with the KL loss is giving us the total loss. So, sum of 0.2643 plus 3.42×10^{-5} is giving us the total loss of 0.2644. So, this is a very minimal loss when compared to the normal uh shallow autoencoder or a deep autoencoder. So, now coming to the reconstruction of the test images, uh what we can do is we can take any 10 sample images, and let us see uh what happens at the reconstruction end. What may be the shape of this reconstructed data? Is it same as the latent space, or is it same as the input data shape? So, that experimentation we are going to do here under reconstruction of test images.

[28:23] So, how do we reconstruct this? Predict the test data. On 10 different samples, let us take 10 different samples. Uh so, encoder doing the prediction of the on the test data, and it is able to give us Z based on Z mean and Z variance. Now, do the reconstruction, which is completely uh covered by this decoder, which is doing the prediction on the latent space that is said. So, let us see the reconstructed image shape. So, reconstructed image shape is 10×784 . 10 is because we are taking example of 10 different samples. We are trying to reconstruct 10 different samples. So, that's why 10 different samples here.

[29:06] But, we are getting back the original shape. Even though it is acting on six different input variables or latent variables, even then we are getting the reconstructed image same as the input image, which is 784 input variables. Now, coming to the last part of this implementation, plotting the original data versus the reconstructed data. So, we make use of this figure function, which makes use of the argument of fixed size in order to control the width and the height of the height of the image. So, 20 in in width and 4 in in height.

[29:43] Let me take 10 different samples for plotting. So, for I in the range 10, so the index varies from 0 to 9 here. Let us have the original image in the subplot, which is divided into two rows and 10 different columns. So, I plus one. I plus one means every iteration it varies from zero. 0 plus one we will get one. 1 plus one will get two. So, likewise we are varying the positions here. We need 10 different positions in order to save 10 different original image. Same thing to do with the reconstruction part also.

[30:22] Once again, we have two rows and 10 columns and all the reconstructed images. So, coming back to original image, we're trying to show the X test data, which has I different samples of data that is 10 samples, which is again reshaped into 28 cross 28. Just like the original input shape. Uh let the color map be gray. Let there be uh no access name. Let uh let us not give any name for the access, x-axis or y-axis. That's why we are trying to keep it off. Okay.

[30:55] And coming to the reconstruction part, let us show the reconstructed image for the same uh samples, that is I different samples. Let this reshape that reconstructed data. We have already seen reconstructed shape is 784. But let us try to reshape that so that it instead of it appearing as an array of uh uh 784 values. Instead of that, we can have 28 values, pixel values in width and 28 pixel values in height. Let the color map be gray and uh let there no be uh you know access name given. Let there no be uh not be any values or names given to the x as well x-axis as well as the y-axis.

[31:37] So, let us show this original as well as the reconstructed image. So, in the first row, we can see all the original images and in the second row, we are able to see all the reconstructed images. So, what is a drawback here is even though we are trying to get the reconstructed image for seven, the image appears something like three or maybe nine. Same way, if we take the original image as two, the reconstructed image what we have for the that image is once again like three, the number three or maybe number nine. So, this is one of the drawback in this case because we are not considering the input labels or we are not considering the labels for every input image. So, what we are trying to do here is

[32:27] we have taken z, which is a latent vector dependent on mu plus the standard deviation into epsilon. And this probability distribution is based on whatever Z we are

calculating only on the input data. But whatever probability we are considering Z on the input data should also have been dependent on Y . So this is missing in this case. That's why we are not able to reconstruct the image properly. So this kind of drawback can be solved in case of a conditional autoencoder. So in our next section, we'll be seeing how a conditional variational autoencoder overcomes all the drawbacks that we have seen in case of a variational autoencoder.

[33:18] Thank you.